

P2549R0

std::unexpected<E> should have error() as member accessor

Published Proposal, 2022-02-13

This version:

<https://wg21.link/P2549R0>

Author:

[Yihe Li](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Source:

github.com/Mick235711/wg21-papers/blob/main/P2549R0.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

1	Revision History
1.1	R0
2	Motivation
3	Design
3.1	Alternative Design
3.1.1	But we are already badly inconsistent!
3.1.2	Conversion operator
3.1.3	No member accessor
3.2	Target Vehicle
3.3	Feature Test Macro
4	Implementation & Usage Experience
4.1	Outcome v2

4.2 Boost.LEAF

5 Wording

- 5.1 20.9.3.2 Class template `unexpected` [expected.un.object]
 - 5.1.1 20.9.3.2.1 General [expected.un.object.general]
 - 5.1.2 20.9.3.2.3 Observers [expected.un.obs]
 - 5.1.3 20.9.3.2.5 Equality operator [expected.un.eq]
- 5.2 20.9.6 Class template `expected` [expected.expected]
 - 5.2.1 20.9.6.2 Constructors [expected.object.ctor]
 - 5.2.2 20.9.6.4 Assignment [expected.object.assign]
 - 5.2.3 20.9.6.7 Equality operators [expected.object.eq]
- 5.3 20.9.7 Partial specialization of `expected` for `void` types [expected.void]
 - 5.3.1 20.9.7.2 Constructors [expected.void.ctor]
 - 5.3.2 20.9.7.4 Assignment [expected.void.assign]
 - 5.3.3 20.9.7.7 Equality operators [expected.void.eq]

References

Normative References

Informative References

[P0323R11] (will be update to R12 once that revision published) introduced class template `std::expected<T, E>`, which is a vocabulary type that contains an expected value of type `T` or an error `E`. Similar to `std::optional<T>`, `std::expected<T, E>` provided member function `value()` and `error()` in order to allow access to the contained value or error type. The proposal also includes the auxillary type `std::unexpected<E>` to wrap the error type, to both disambiguating between value and error types, and also introduce explicit marker for returning error (unexpected outcome) types. The introduction of the wrapper type allows both `T` and `std::unexpected<E>` to be implicitly convertible to `std::expected<T, E>`, and thus allows the following usage:

```
std::expected<int, std::errc> svtoi(std::string_view sv)
{
    int value{0};
    auto [ptr, ec] = std::from_chars(sv.begin(), sv.end(), value);
    if (ec == std::errc{})
    {
        return value;
    }
    return std::unexpected(ec);
}
```

However, even though `std::unexpected<E>` is simply a wrapper over `E`, we need to use its member method `value()` to access the contained error value. The name of the member method is inconsistent with the `std::expected<T, E>` usage and intuition, so this proposal seeks to correct the name of the member access method to `error()`.

[P0323R11] is adopted for C++23 at the February 2022 WG21 Plenary, so this proposal also targets C++23 to fix this.

§ 1. Revision History

§ 1.1. R0

Initial revision.

§ 2. Motivation

Consistency among library vocabulary types is important and makes user interaction intuitive, and since `std::expected<T, E>` is specifically based on and extends `std::optional<T>` [N3793], it is especially important to maintain a similar interface between `std::optional<T>` and `std::expected<T, E>`, and also within the `expected` design. By this way, users will not be surprised if they switch between different sum types.

We can have a comparison on the various member access method of the `optional` and `expected` interface:

Member	Return Type	<code>std::optional<T></code>	<code>std::expected<T, E></code>	<code>std::unexpected<E></code>	<code>std::bad_expected_access<E></code>
(Normal) Value	<code>T</code>	<code>value()</code> <code>operator*</code> <code>operator-></code>	<code>value()</code> <code>operator*</code> <code>operator-></code>	N/A	N/A
Unexpected Outcome (Error)	<code>E (std::nullopt_t)</code>	N/A	<code>error()</code>	<code>value()</code>	<code>error()</code>

We can see that the only outlier in this table is `std::unexpected<E>::value()`, which is both inconsistent with `std::expected<E>` and `std::bad_expected_access<E>` that also (possibly) holds an error value, and also inconsistent with other standard library types providing `value()`, including `std::optional<T>` and `std::expected<T, E>`. These types all provide `value()` to access the normal value they hold, and often have preconditions (or throw exceptions if violated) that they holds a value instead of an error.

Provide `error()` instead of `value()` for `std::unexpected<E>` has several benefits:

1. **Consistency:** Both consistent with `std::expected<T, E>` and `std::bad_expected_access<E>` to provide `error()` to return error value, and also reduce inconsistency with other `value()`-providing types that have different preconditions.
2. **Generic:** Same name means more generic code is allowed. For example, generic code can do `e.error()` on any (potentially) error-wrapping types to retrieve the error, this includes `std::expected<T, E>`, `std::unexpected<E>`, `std::bad_expected_access<E>`, and possible further error-handling types like `std::status_code` [P1028R3] and `std::error` [P0709R4].
3. **Safety & Intuitive:** Other `value()` type often have different preconditions, for example throwing when the type does not holds a normal value, or (worse) have narrow contract and UB on abnormal call. Passing the current `std::unexpected<E>`-wrapped type to interface expecting the normal `value()` semantics can be surprising when leading to runtime exception or (worse) UB.

Side note: you can even smell the inconsistency when many of the wording of equality operator between `expected<T, E>` and `unexpected<E>` in [P0323R11] contains clause such as `x.error() == e.value()`.

§ 3. Design

§ 3.1. Alternative Design

This section lists the alternative choices and possible arguments against this proposal that has been considered.

§ 3.1.1. But we are already badly inconsistent!

Some may argue that the intensive use of `value()` across the library is already inconsistent, and we do not need to keep it consistent.

I would argue that most use of `value()` member function across the standard library adhere to the tradition, aka return the normal "value" that the type is holding. Functions like `std::chrono::leap_second::value()` can be thought as an extended definition of "value": leap second can hold either `+1s` or `-1s` as value. The only case that I agree are related is the C++11 `std::error_code/std::error_condition` pair and their `value()` member that returns the error code, which seems to return an error(-related) value. However, I want to point out that `value()` is not really the "error value" or "unexpected outcome" of these types, since this is the expected outcome (or "normal value") or an `std::error_code`. Furthermore, `value()` is not really the whole "error" contained in these types, since these two types consists of `value()` plus `category()`. Only `value()` cannot represent a unique error, and should not be taken as the "error representation".

§ 3.1.2. Conversion operator

The other standard library wrapper type, `std::reference_wrapper<T>`, provided an (implicit) conversion operator to `T&`, its wrapped value. This leads to thoughts on whether `std::unexpected<E>` should simply provide an (implicit or explicit) conversion operator to `E` as its member access method.

A similar choice had been facing the designer of `std::optional<T>`, and their decision (later inherited by `std::expected<T, E>`) is to reject: ([N3672], 7.9)

We do not think that providing an implicit conversion to T would be a good choice. First, it would require different way of checking for the empty state; and second, such implicit conversion is not perfect and still requires other means of accessing the contained value if we want to call a member function on it.

I think that this reasoning also applies here. Even if it is implicit, conversion operator is not perfect (call member functions), and we still need `static_cast` or other member accessors to do that. Also, there seems to be no benefit in providing such conversion (besides, `std::unexpected<E>` is just intended as a "trampoline" for constructing an `std::expected<T, E>`, it is not intended to be used extensively/on its own). Therefore I rejected this option.

§ 3.1.3. No member accessor

The above discussion leads to the consideration: since `std::unexpected<E>` is just meant as a "trampoline", does it need a member accessor at all? Besides, the intended usage is just `return std::unexpected(some_error);`, provide a member accessor does not seems to help this use case at all.

This is an interesting point. Also, one of [P0323R11]'s referenced implementation [viboes-expected] does this: its `std::experimental::fundamental_v3::unexpected<E>` type have no accessor at all. However, providing an accessor does not seem to do any harm, and may have interesting use cases that I'm not aware of. Therefore I do not propose this, but will not go against changing the proposal to this direction if L(E)WG favor this.

§ 3.2. Target Vehicle

This proposal targets C++23. I'm aware that the design freeze deadline of C++23 is already passed, but I think this can be classified as an improvement/fix over the defect in `std::expected<T, E>`. Furthermore, this proposal will be a huge breaking change (that makes it simply unviable to propose) after C++23.

§ 3.3. Feature Test Macro

As long as the proposal lands in C++23, I don't think there is a need to change any feature test macro. However, if L(E)WG feels there is a need, then I suggest bumping `__cpp_lib_expected` to the date of adoption (definitely larger than 202202L).

§ 4. Implementation & Usage Experience

The referenced implementation of [P0323R11] all implement the interface of original proposal (except [viboes-expected] mentioned above). This section thus investigated several similar implementations.

§ 4.1. Outcome v2

[outcome-v2] is a popular library invested in a set of tools for reporting and handling function failures in contexts where *directly* using C++ exception handling is unsuitable. It is both provided as Boost.Outcome and the standalone GitHub repository, and also having an experimental branch that is the basis of [P1095R0] and [P1028R3] `std::status_code`. The library provided `result<T, E, Policy>` and `outcome<T, EC, EP, Policy>` types that represent value/error duo type, just like `std::expected<T, E>`, with the difference in interface and also `outcome` can hold both `EC` and `EP` (error code and exception (pointer)). The design of `result<T, E>` also deeply influences [P0323R11], and the final adopted design of `std::expected<T, E>` is very similar to what `outcome::result<T, E>` provides.

One of the main design differences is that `result<T, E>` can be implicitly constructed from both `T` and `E`, while `std::expected<T, E>` can only be implicitly constructed from the former. For this reason, `result<T, E>` does not allow for `T` and `E` to be the same, and also does not provide `operator*` and `operator->` accessor. Thus, there are wrappers for both success and failure value for construction, and `success_type<T>` wraps a success `T`, while `failure_type<EC, EP>` wraps an unexpected `E` (or `EC` and `EP`). Their accessors are: (the `assume_*` narrow-contract accessors and `failure()` are not shown)

Member	Return Type	<code>result<T, E></code>	<code>outcome<T, EC, EP></code>	<code>success_type<T></code>	<code>failure_type<EC, EP></code>
(Normal) Value	<code>T</code>	<code>value()</code>	<code>value()</code>	<code>value()</code>	N/A

Member	Return Type	<code>result<T, E></code>	<code>outcome<T, EC, EP></code>	<code>success_type<T></code>	<code>failure_type<EC, EP></code>
Unexpected Outcome (Error)	<code>E</code> (or <code>EC</code> and <code>EP</code>)	<code>error()</code>	<code>error()</code> <code>exception()</code>	N/A	<code>error()</code> <code>exception()</code>

We can see that Outcome v2 is pretty consistent in member accessor, and especially its `failure_type<EC, EP>` provides `error()` and `exception()`, not `value()`. Also note that the default exception being thrown, `bad_result_access` and `bad_outcome_access`, does not holds the error/exception value at all. There is a `bad_result_access_with<E>` for consistency with `std::expected<T, E>`.

§ 4.2. Boost.LEAF

Lightweight Error Augmentation Framework (LEAF), or [\[Boost.LEAF\]](#), is a lightweight error handling library for C++11. It is intended to be an improved version of Outcome, by eliminating branchy code and remove error type from `result<T, E>` signature. The author describe it as

LEAF is designed with a strong bias towards the common use case where callers of functions which may fail check for success and forward errors up the call stack but do not handle them. In this case, only a trivial success-or-failure discriminant is transported. Actual error objects are communicated directly to the error handling scope, skipping the intermediate check-only frames altogether.

The main type for LEAF is `leaf::result<T>`, which is again a counterpart of `std::expected<T, E>` and `outcome::result<T, E>`, but with `E` eliminated from signature. Unexpected results are produced by `leaf::new_error(some_error)`, which returns an `leaf::error_id` object that can be converted to an unexpected `leaf::result<T>`. There is also a `leaf::error_info` that is used as the generic error type receiver for functions such as `leaf::try_catch`. The member accessor is:

Member	Return Type	<code>leaf::result<T></code>	<code>leaf::error_info</code>
(Normal) Value	<code>T</code>	<code>value()</code>	N/A
Unexpected Outcome (Error)	<code>leaf::error_id</code>	<code>error()</code>	<code>error()</code>

(Notice that `leaf::error_id` is the final error (unexpected outcome) type, its `value()` is similar to that of `std::error_code`, which does not return an "unexpected outcome", but instead return an error ID for the alternative description of `leaf::error_id`, which actually fits into my reasoning of returning "value".) Again we can see a consistency here.

§ 5. Wording

The wording below is based on [\[N4901\]](#) plus the editorial PR of P0323R12. This will be rebased onto post-February Working Draft in the next revision, with hopefully no change of section name.

§ 5.1. 20.9.3.2 Class template `unexpected` [expected.un.object]

§ 5.1.1. 20.9.3.2.1 General [expected.un.object.general]

```
namespace std {
    template<class E>
    class unexpected {
    public:
        constexpr unexpected(const unexpected&) = default;
        constexpr unexpected(unexpected&&) = default;
        template<class... Args>
            constexpr explicit unexpected(in_place_t, Args&&...);
        template<class U, class... Args>
            constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);
        template<class Err = E>
            constexpr explicit unexpected(Err&&);

        constexpr unexpected& operator=(const unexpected&) = default;
        constexpr unexpected& operator=(unexpected&&) = default;

        constexpr const E& valueerror() const& noexcept;
        constexpr E& valueerror() & noexcept;
        constexpr const E&& valueerror() const&& noexcept;
        constexpr E&& valueerror() && noexcept;

        constexpr void swap(unexpected& other) noexcept(see below);

        template<class E2>
            friend constexpr bool operator==(const unexpected&, const unexpected<E2>&);

        friend constexpr void swap(unexpected& x, unexpected& y) noexcept(noexcept(x.swap(y)));

    private:
        E val; // exposition only
    };

    template<class E> unexpected(E) -> unexpected<E>;
}
```

§ 5.1.2. 20.9.3.2.3 Observers [expected.un.obs]

```
constexpr const E& valueerror() const& noexcept;
constexpr E& valueerror() & noexcept;
```

Returns: `val`.

```
constexpr E&& valueerror() && noexcept;  
constexpr const E&& valueerror() const&& noexcept;
```

Returns: `std::move(val)`.

§ 5.1.3. 20.9.3.2.5 Equality operator [expected.un.eq]

```
template<class E2>  
    friend constexpr bool operator==(const unexpected& x, const unexpected<E2>& y);
```

Mandates: The expression `x.valueerror() == y.valueerror()` is well-formed and its result is convertible to `bool`.

Returns: `x.valueerror() == y.valueerror()`.

§ 5.2. 20.9.6 Class template `expected` [expected.expected]

§ 5.2.1. 20.9.6.2 Constructors [expected.object.ctor]

```
template<class G>  
    constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);  
template<class G>  
    constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is `true`.

Effects: Direct-non-list-initializes `unex` with `std::forward<GF>(e.valueerror())`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

§ 5.2.2. 20.9.6.4 Assignment [expected.object.assign]

```
template<class G>  
    constexpr expected& operator=(const unexpected<G>& e);  
template<class G>  
    constexpr expected& operator=(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints:

- `is_constructible_v<E, GF>` is true; and
- `is_assignable_v<E&, GF>` is true; and
- `is_nothrow_constructible_v<E, GF> || is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is true.

Effects:

- If `has_value()` is true, equivalent to:

```
reinit-expected(unex, val, std::forward<GF>(e.valueerror()));  
has_val = false;
```

- Otherwise, equivalent to: `unex = std::forward<GF>(e.valueerror());`

Returns: `*this`.

§ 5.2.3. 20.9.6.7 Equality operators [expected.object.eq]

```
template<class E2> friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

Mandates: The expression `x.error() == e.valueerror()` is well-formed and its result is convertible to `bool`.

Returns: `!x.has_value() && static_cast<bool>(x.error() == e.valueerror())`.

§ 5.3. 20.9.7 Partial specialization of `expected` for `void` types [expected.void]

§ 5.3.1. 20.9.7.2 Constructors [expected.void.ctor]

```
template<class G>  
    constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);  
template<class G>  
    constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is true.

Effects: Direct-non-list-initializes `unex` with `std::forward<GF>(e.valueerror())`.

Postconditions: `has_value()` is false.

Throws: Any exception thrown by the initialization of `unex`.

§ 5.3.2. 20.9.7.4 Assignment [expected.void.assign]

```
template<class G>
constexpr expected& operator=(const unexpected<G>& e);
template<class G>
constexpr expected& operator=(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is true and `is_assignable_v<E&, GF>` is true.

Effects:

- If `has_value()` is true, equivalent to:

```
construct_at(addressof(unex), std::forward<GF>(e.value_error()));
has_val = false;
```

- Otherwise, equivalent to: `unex = std::forward<GF>(e.value_error());`

Returns: `*this`.

§ 5.3.3. 20.9.7.7 Equality operators [expected.void.eq]

```
template<class E2>
constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

Mandates: The expression `x.error() == e.value_error()` is well-formed and its result is convertible to `bool`.

Returns: `!x.has_value() && static_cast<bool>(x.error() == e.value_error())`.

§ References

§ Normative References

[P0323R11]

JF Bastien, Jonathan Wakely, Vicente Botet. [std::expected](https://wg21.link/p0323r11). 16 November 2021. URL: <https://wg21.link/p0323r11>

§ Informative References

[Boost.LEAF]

Emil Dotchevski. *Lightweight Error Augmentation Framework written in C++11*. URL: <https://boostorg.github.io/leaf/>

[N3672]

F. Cacciola, A. Krzemiński. *A proposal to add a utility class to represent optional objects (Revision 4)*. 19 April 2013. URL: <https://wg21.link/n3672>

[N3793]

F. Cacciola, A. Krzemiński. *A proposal to add a utility class to represent optional objects (Revision 5)*. 3 October 2013. URL: <https://wg21.link/n3793>

[N4901]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 22 October 2021. URL: <https://wg21.link/n4901>

[OUTCOME-V2]

Niall Douglas. *Standalone Outcome v2: Lightweight Error Handling Framework*. URL: <https://github.com/ned14/outcome>

[P0709R4]

Herb Sutter. *Zero-overhead deterministic exceptions: Throwing values*. 4 August 2019. URL: <https://wg21.link/p0709r4>

[P1028R3]

Niall Douglas. *SG14 status_code and standard error object*. 12 January 2020. URL: <https://wg21.link/p1028r3>

[P1095R0]

Niall Douglas. *Zero overhead deterministic failure - A unified mechanism for C and C++*. 29 August 2018. URL: <https://wg21.link/p1095r0>

[VIBOES-EXPECTED]

Vicente J. Botet Escriba. *viboes's Implementation of LFTSv3 std::expected<T, E>*. URL: <https://github.com/viboes/std-make/blob/master/include/experimental/fundamental/v3/expected2/expected.hpp>